

PYTHON

1. Python Introduction:-

- (Python introduced by Guido Van Rossum.)
- (Released under GNU License.)
- (Its interpreted language, OOPS Oriented as well.)
- (Python Supports Functional Programming, Structure & Programming methods as well.)
- (Now a days python is for beginner's language.)
- (Very famous - extensive Coats of Libraries)
- (Simple Syntax)
- (Has ability to develop web, mobile, games and Stand alone Software's as well.)
- (It supports Scripting as well.)
- (Written in C.)

2. How To Download & Install pycharm?

- Download Pycharm - Search Chrome
- Click 1st result.
- Download Community pycharm
- Open file and install pycharm.
- Create Desktop shortcut ☒ 64-bit launcher
- Then click next to install.
- after read an license term.
- Open pycharm → then click a symbol Create Project → And installing components.

PYTHON

→ File → new scratch file → DTD → python
Project → v new → script → new → python
file → Hello.py

Program :-

```
Printf ("Hello world")
```

Output :-

Hello world

Process finished with exit code 0.

3. How To Download & Install Python?

→ Search python download

→ Click 1st link

→ Download python 3.8.5 (download latest version)

→ python 3.8.5 (32-bit setup)

→ tick the checkbox

☒ Install launcher for all users (recommended)

☒ Add python 3.8 to path.

→ Then click Install now.

→ Then close.

→ Open python 3.8.5 (32-bit)

→ And Type print ("Hello world")

→ Output - Hello world.

4. python IDE for Windows:-

→ IDE's - Integrated Development Environment.

- Download Sublime text 3 for using purpose.
- Download Visual Studio code for using purpose.
- Download hyper comment prompt.
- Open python 3.7 (32-bit)

5. Python Environment :

- (Download Windows Version)
- (Then select python - 3.7.2.exe)
- then tick all column for optional features.
- (Then click next and add path on environmental variable add and type last in (;) semicolon.)
- (Open command prompt And type python -- Version.)
- (Type python show version and another details)
- (>>> exit())

6. Python Hello World Program :-

- (Open Command Prompt)
- (Type python)
- (And print ("python Tamil Tutorial"))
- (Output - python Tamil Tutorial)
- (input ())
- (Output - 'a')
- (Print ("Hello world"))
- (Output - Hello world)
- (input ("enter your name"))
- (Output - enter your name! Oropi
'Oropi')

→ Open Visual Studio Code and create file
index.py)

→ Then Type Print ("Welcome to the python Tamil
Tutorial")

→ Press Ctrl + ~ - Terminal

→ To go on path file for (saved python
file Index.py) then Type python index.py

→ Output - Welcome to the python Tamil Tutorial

7. Python DataTypes :-

→ Open Command Prompt)

→ Python is a dynamic type variable.

→ >>> a = 10

>>> a

10

→ >>> a = 'tutorial'

>>> a

'tutorial'

→ >>> int a = 0

File "<stdin>", line 4: 'a' - int

int a = 0

^

Syntax Error: invalid Syntax

>>> a

'tutorial'

→ >>> a = "asdf"

>>> a

'asdf'

Python Comments

>>> A = "asd"

>>> A

'asd'

→ >>> age = input("Enter your name")

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

NameError: Name 'input' is not defined.

>>> age = input("Enter your name")

Enter your name 123

>>> age

'123'

→ >>> age = int(input("Enter your age"))

Enter your age 3

>>> age

3

>>> type(age)

<class 'int'> // Type casting

>>> print("your age is" + age)

File "<stdin>", line 1

print("your age is" + age)

SyntaxError: invalid syntax

>>> print("your age is" + str(age))

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

TypeError: Can only concatenate str (not "int") to str

>>> print("your age is" + str(age))

you are 3
>>>
8. Python Comments:-

→ Open index.py

→ Class Student:

def __init__(self);

pass

def get-Student(self);

pass

def add-Student(self);

pass

def delete-Student(self);
pass

def total_and_average(self);

Depends on the total numbers of the subjects

total will get vary

total +

Average of the Students calculated based on the

number of subjects.

avg 15

→ # This is Comment // Single line comment

→ # block comment

block comment block comment

block comment

→

block comment

block comment block comment

block comment

'''

9. Python If else :-

- Used when you wanted to check the condition.
- Validates to Boolean type always
- Syntax :-

```
if (condition):
```

```
.....
```

```
else:
```

```
.....
```

- Another condition is if else below is the syntax.

```
if (condition):
```

```
-----
```

```
elif:
```

```
else:
```

```
-----
```

10. Python If else Program :

- Open index.py
- `mark = int(input("Enter your mark"))`
- `if (mark > 30):`
- `Print("pass")`
- `else:`
- `Print("Fail")`

→ Output - Enter your mark 30.

Fail

→ `mark = int(input("Enter your mark"))`

if (mark > 0 and mark < 30);

Print ("fail")

elif (mark < 60):

Print ("pass")

else:

Print ("distinction")

→ Output - Enter your mark 70
distinction

Enter your mark 55
pass

Enter your mark 33
pass

11. Python for loop:-

→ Used when you wanted to perform some action repeatedly.

→ Syntax:-

```
for (int i=0; i<10; i++) {  
    Printf(i);  
}
```

```
for (i in datas):  
    Print(i)
```

→ Code sample:-

```
for (name in range(0, 10)):  
    Print(name)
```

12. Python for loop Program:

→ Open Index.py

→ number = int (input ("Enter a number"))

for target_list in range (0, number):

Print (target_list)

→ Output :- Enter a number 10

0

1

2

3

4

5

6

7

8

9

Can use for target_list in range (number):

0

1

2

3

4

5

6

7

8

9

(Same output)

→ for target_list in range (0, number):

starting number
0 to 9

13. Python for increment Program:

→ number = int (input ("Enter a number"))

for target_list in range (0, number, 2):

Print (target_list)

→ Output - Enter a number 10

0

2

4

6

8

→ number = int (input ("Enter a number"))

for target_list in range(2, numbers.4):

Print (target_list)

→ Output -  Once run this program (Output not confirmed.)

14. Python String :-

→ String are collection of characters

→ Can be access each and every character of a string using it's index.

→ Symbols used for

→ `[]` - Slice

→ `%` - Formatting

→ `[:]` - Range

→ `+` - Concatenate

→ `*` - Repetition

→ `in` - Validate to Boolean

15. Python String Program :-

→ `# Cj`

`# %`

`# [:]`

`# +`

`# *`

`# in`

`# not in`

tut = "python tamil tutorial"

Print (len(tut))

→ Output - Python index.py

→ tut = "python tamil tutorial"

Print (tut [5]) * "in")

→ Output - python index.py

n

→ tut = "python tamil tutorial"

Print (tut [0:5])

→ Output - python index.py

pytho

Can use print (tut [0:6]) :-

Output - python index.py

python

→ tut = "python tamil tutorial"

Print ("my name is %s and my age is %d"

%('Gopi', 27))

→ Output - Python index.py

My name is Gopi and my age is 27

→ tut = "python tamil tutorial"

Print ("Hello" + "world")

→ Output - python index.py

Hello world

→ tut = "python tamil tutorial"

Print (tut + " youtube") // use space on youtube

→ Output - Python index.py

python tamil tutorial youtube

→ tut = "python tamil tutorial"

Print ("hi" * 3)

→ Output - hihhi

→ tut = "python tamil tutorial"

Print ('p' in ^{tut}~~tut~~) // use small p

→ Output - python index.py

True

→ tut = "python tamil tutorial"

Print ('P' in ~~tut~~ tut)

→ Output - python index.py

False

// Not in Caps on tut

→ tut = "python tamil tutorial"

if ('g' in tut):

Print (True)

else :

Print (False)

→ Output - python index.py

False

→ tut = "python tamil tutorial"

if ('g' not in tut):

Print (True)

else :

Print (False)

→ Output - Python index.py

True

→ tut = "python tamil tutorial"

if ('tamil' not in tut):

Print (True)

else:

Print (False)

→ Output - python index.py

False

Can use in :-

Output - python index.py

True

→ tut = "python tamil"

Print (tut.strip())

Output - python tamil

16. Python List Theory :-

→ Lists or most common in Python

→ Can access elements using index

→ Can have any type of data also

→ Can have nested list as well.

→ We can do slicing, concatenate, repetition, in check and iterations as well.

→ Built in methods min, max, cmp, append, count, insert, pop, removes, sort.

17. Python List Program :-

→ lst = [1, 2, 3, 'a', 'b', 'c']

Print (lst)

→ Output - python index.py

[1, 2, 3, 'a', 'b', 'c']

→ `lst = [1, 2, 3, 'a', 'b', 'c']`

`Print (type (lst))`

→ Output :- python index.py

`<class 'list'>`

→ `lst = [1, 2, 3, 'a', 'b', 'c']`

`Print (lst [3])`

→ Output - python index.py

`a`

→ `lst = [1, 2, 3, 'a', 'b', 'c']`

`Print (lst [:3])`

→ Output - python index.py

`[1, 2, 3]`

→ `lst = [1, 2, 3, 'a', 'b', 'c']`

`Print (lst [3:])`

→ Output :- python index.py

`['a', 'b', 'c']`

→ `lst = [1, 2, 3, 'a', 'b', 'c']`

`Print (len (lst))`

→ Output :-

`6`

→ `lst = [1, 2, 3, 'a', 'b', 'c']`

`lst+1 = [0.1, 0.2, 0.4]`

`con = lst + lst+1`

`Print (con)`

→ Output :- python index.py

[1, 2, 3, 'a', 'b', 'c', 0.1, 0.2, 0.4]

→ l1 = [1, 2, 3, 'a', 'b', 'c']

l2 = [0.1, 0.2, 0.4]

con = l1 + l2

for i in l1:

print(i)

→ Output :- python index.py

1

2

3

'a'

'b'

'c'

→ l1 = [1, 2, 3, 'a', 'b', 'c']

l2 = [0.1, 0.2, 0.4]

con = l1 + l2

a = l1[4] + 4

print(a)

→ Output :- python index.py

['hi', 'hi', 'hi', 'hi']

→ l1 = [1, 2, 3]

l2 = ['a', 'b', 'c']

print(max(l1))

→ Output :- python index.py

3

→ l1 = [1, 2, 3]

l2 = ['a', 'b', 'c']

print(min(l1))

→ Output - python index.py

1

→ lst = [1, 2, 3]

lst1 = ['a', 'b', 'c']

lst.append(4)

Print(lst)

→ Output :- python index.py

[1, 2, 3, 4]

→ lst = [1, 2, 3]

lst1 = ['a', 'b', 'c']

lst.append(5)

Print(lst)

→ Output - python index.py

[1, 2, 3, 5]

→ lst = [1, 2, 3]

lst1 = ['a', 'b', 'c']

lst.append(5)

lst.append(6)

Print(lst)

→ Output - python index.py

[1, 2, 3, 5, 6]

→ lst = [1, 2, 3]

lst1 = ['a', 'b', 'c']

lst.index(5)

Print (lst)

→ Output - python index.py

Traceback (most recent call last):

File "index.py", line 4, in <module>

lst.index(5)

Value Error : 5 is not in list

→ lst = [1, 2, 3]

lst1 = ['a', 'b', 'c']

lst.index(2)

Print (lst)

→ Output - python index.py

[1, 2, 3]

→ lst = [1, 2, 3]

lst1 = ['a', 'b', 'c']

a = lst.index(1)

Print (a)

→ Output - python index.py

0

Can use a = lst.index(3)

Output - 2

→ lst = [1, 2, 3]

lst1 = ['a', 'b', 'c']

a = lst.remove(1) // not return element

Print (a)

Print (lst)

→ Output → python index.py

[1, 2]

→ lst = [1, 2, 3]

lst = ['a', 'b', 'c']

a = lst.pop(2)

Print(a)

Print(lst)

→ Output - python index.py

3

[1, 2]

→ Pop - It returns ^{element} (Value) (Interview question)

→ lst = [4, 5, 6, 1, 2, 4]

a = lst.reverse()

Print(a)

Print(lst)

→ Output - python index.py

None

[4, 2, 1, 6, 5, 4]

→ lst = [4, 5, 6, 1, 2, 4]

a = lst.sort()

Print(a)

Print(lst)

→ Output - python index.py

None

[1, 2, 4, 4, 5, 6]

→ `lst = [4, 5, 6, 1, 2, 4]`

`a = lst.sort(reverse=True)`

`Print(a)`

`Print(lst)`

→ Output - python index.py

None

`[6, 5, 4, 4, 2, 1]`

→ discuss compare methods.

18. python Tuples Theory :-

→ List are most common in Python

→ Can access elements using index

→ Can have any type of data also

→ It's immutable.

→ it's use()

→ We can do slicing, Concatenate, repetition, in check and iterations as well.

→ Built in methods min, max, cmp, tuple.

→ List is mutable.

→ Tuples are immutable. (not change elements)

19. Python Tuples Program :-

→ `tup = (1, 2, 3, 'a', 'b', 'c')`

`Print(tup)`

→ Output :- python index.py

`(1, 2, 3, 'a', 'b', 'c')`

→ `tup = (1, 2, 3, 'a', 'b', 'c')`

`Print(type(tup))`

→ Output - python index.py

`<class 'tuple'>`

→ tup = (1, 2, 3, 'a', 'b', 'c')

tup 4 = (1, 3, 2, 1)

Print (tup + tup 4)

→ Output - python index.py

(1, 2, 3, 'a', 'b', 'c', 1, 3, 2, 1)

→ tup = (1, 2, 3, 'a', 'b', 'c')

tup 4 = (1, 3, 2, 1)

for i in tup:

Print (i)

Print (tup + tup 4)

→ Output - python index.py

1
2
3
a
b
c

(1, 2, 3, 'a', 'b', 'c', 1, 3, 2, 1)

→ tup = (1, 2, 3, 'a', 'b', 'c')

tup 4 = (1, 3, 2, 1)

rep = ('hi') * 4

Print (rep)

→ Output - python index.py

hihihihi

→ tup = (1, 2, 3, 'a', 'b', 'c')

tup 4 = (1, 3, 2, 1)

rep = ('hi') * 4

a = tup.append('5')

Print (aep)

→ Output - Error occurred // no change in tuple

Can use insert

ERROR occurred // No change element

→ tup = (1, 2, 3, 'a', 'b', 'c')

tup1 = (1, 3, 2, 1)

Print (max (tup1))

→ Output - python index.py

3

→ tup = (1, 2, 3, 'a', 'b', 'c')

tup1 = (1, 3, 2, 1)

Print (min (tup1))

→ Output - python index.py

1

→ tup = (1, 2, 3, 'a', 'b', 'c')

tup1 = (1, 3, 2, 1)

Cmp () // Compare method is not support in Python 3 version. So no output occurred.

20. Python Dictionary Theory:-

→ Dictionary are Key Value pair

→ Dictionaries are represents as {"Key": "value"}

→ Access dictionary using Keys

→ It's mutable

→ Key must be unique

→ Built in methods like get, keys, copy, clear

→ Dict = {

"Name" : "Gopi",

"No" : 1

}

21. Python Dictionary Program:-

→ dic = { "name" : "john", "Std" : 10, "maj-sub" :
"Computer" }

Print (dic)

→ Output - python index.py

{ 'name' : 'john', 'Std' : 10, 'maj-sub' :
'Computer' }

→ dic = { "name" : "john", "Std" : 10, "maj-sub" :
"Computer" }

Print (dic ["name"])

→ Output : python index.py

John

→ dic = { "name" : "john", "Std" : 10, "maj-sub" :
"Computer" }

dic ["Std"] = 11

Print (dic)

→ Output - python index.py

{ 'name' : 'john', 'Std' : 11, 'maj-sub' : 'Computer' }

dic = {"name": "john", "std": 10, "maj-sub": "computer"}

```
del dic ["maj-sub"]
```

```
Print(dic)
```

→ Output - python index.py

```
{'name': 'john', 'std': 10}
```

→ dic = {"name": "john", "std": 10, "maj-sub": "computer"}

```
a = dic.get("name")
```

```
Print(a)
```

→ Output - python index.py

John

→ dic = {"name": "john", "std": 10, "maj-sub": "computer"}

```
a = dic.get("name a")
```

```
Print(a)
```

→ Output - python index.py

None

→ dic = {"name": "john", "std": 10, "maj-sub": "computer"}

```
a = dic.clear()
```

```
Print(a)
```

```
Print(dic)
```

→ Output - python index.py

None

```
{}
```

```
dic = {"name": "john", "std": 10, "maj-sub":  
"Computer"}
```

```
dic1 = dic.copy()
```

```
dic["name"] = "jhonny"
```

```
Print(dic)
```

```
Print(dic1)
```

→ Output - python index.py

```
{'name': 'john', 'std': 10, 'maj-sub': 'Computer'}
```

```
{'name': 'johnd', 'std': 10, 'maj-sub': 'Computer'}
```

```
→ dic = {"name": "john", "std": 10, "maj-sub":  
"Computer"}
```

```
dic1 = dic.copy()
```

```
dic1["name"] = "Jhonny"
```

```
Print(dic)
```

```
Print(dic1)
```

→ Output - python index.py

```
{'name': 'john', 'std': 10, 'maj-sub':
```

```
'Computer'}
```

```
{'name': 'johnd', 'std': 10, 'maj-sub': 'Computer'}
```

```
→ dic = {"name": "john", "std": 10, "maj-sub":  
"Computer", "list": [90, 80, 23]}
```

```
dic1 = dic.copy() // shallow copy
```

```
dic1 ["list"] . append (100)
```

```
Print (dic)
```

```
Print (dic1)
```

→ Output - python index.py

```
{ 'name' : 'john' , 'std' : 10 , 'maj-sub' :  
  'computer' , 'list' : [90, 80, 23, 100]}
```

```
{ 'name' : 'john' , 'std' : 10 , 'maj-sub' :  
  'computer' , 'list' : [90, 80, 23, 100]}
```

```
→ dic = { "name" : "john" , "std" : 10 , "maj-sub" : "compu  
  -ter" , "list" : [90, 80, 23]}
```

```
dic1 = dic . copy()
```

```
dic1 ["name"] = "johny"
```

```
dic1 ["list"] . append 100
```

```
Print (dic)
```

```
Print (dic1)
```

→ Output - python index.py

```
{ 'name' : 'john' , 'std' : 10 , 'maj-sub' : 'computer',  
  'list' : [90, 80, 23, 100]}
```

```
{ 'name' : 'johny' , 'std' : 10 , 'maj-sub' :
```

```
  'computer' , 'list' : [90, 80, 23,  
    100]}
```

```
→ dic = { "name" : "john" , "std" : 10 , "maj-sub" :  
  "computer" , "list" : [90, 80, 23]}
```


dic1 = dic . copy ()

dic1 ["name"] = "johny"

dic1 ["list"]. append (100)

dic1 ["list"]. append (200)

Print (dic)

Print (dic1)

-> Output - python index.py

{'name': 'john', 'std': 10, 'maj-sub':
'computer', 'list': [90, 80, 23,
100, 200]}

{'name': 'johny', 'std': 10, 'maj-sub':
'computer', 'list': [90, 80, 23,
100, 200]}

22. Python Set Theory:-

→ Set is another data structure

→ Represents with {}

→ Used to remove the duplicates

→ Merge two sets with Union method

→ Set can have tuples in it but list and dictionaries are not allowed.

→ Strings can in set.

23. Python Set Program:-

→ Open index.py

→ tut = set ()

Print (tut)

→ Output → python index.py
Set ()

→ tut = Set ('a')

Print (tut)

→ Output → python index.py
{'a'}

→ tut = Set (['a', 'b'])

Print (tut)

→ Output → python index.py
{'a', 'b'}

→ tut = Set (['a', 'b', 'b', 'c']) // remove duplicate element

Print (tut)

→ Output → python index.py
{'c', 'a', 'b'}

→ tut = Set (['a', 'b', 'b', 'c'])

tut.add ('g')

Print (tut)

→ Output → python index.py

{'c', 'a', 'b', 'g'}

→ tut = Set (['a', 'b', 'b', 'c'])

tut1 = Set (['a', 'b', 'b', 'c', 'd'])

tut.union (tut1)

Print (tut1)

→ Output → Python index.py

{'b', 'a', 'c'}

Can insert tut1 = set(['a', 'b', 'b', 'c', 'd'])

Output - Python index.py

{'a', 'b', 'd', 'c'}

→ tut = set(['a', 'b', 'b', 'c'])

tut1 = set(['a', 'b', 'b', 'c', 'd'])

tut.intersection(tut1)

Print(tut1)

→ output - Python index.py

{'d', 'b', 'a', 'c'}

→ tut = set(['a', 'b', 'b', 'c'])

tut.add('g')

Print(tut)

→ Output - Python index.py

{'g', 'c', 'b', 'a'}

→ tut = set(['a', 'b', 'b', 'c'])

tut.add(['g', 'o'])

print(tut)

→ Output → python inden.py

Traceback (most recent call last):

File "inden.py", line 3, in module

tut.add(['g', 'o'])

Type Error: unhashable type: 'list'

:- non-hashable

python

→ tut = set(['a', 'b', 'b', 'c'])

a = "Gopi"

tut.add(a)

Print(tut)

→ Output - python inden.py

['Gopi']

{'b', 'a', 'Gopi', 'c'}

→ tut = set(['a', 'b', 'b', 'c'])

a = ("a", 'a', 'b')

tut.add(a)

Print(tut)

→ Output - python inden.py

{'c', 'b', 'a', ('a', 'a', 'b')}

:- non-hashable

Function

python

22

→ tut = set(['a', 'b', 'c', 'd'])

a = ("a", 'a', 'b')

tut.add(a)

Print(tut)

Print(len(tut))

→ Output — python index.py

{'c', 'b', ('a', 'a', 'b'), 'a'}

4

24. Python Comparison:-

Data structure	Ordered	Mutable	Constructors	Example
List	Yes	Yes	[] or list()	[5, 7, 4, 'yes', 5.7]
Tuple	Yes	No	() or tuple()	(5, 7, 4, 'yes', 5.7)
Set	No	Yes	{ }* or set()	{5, 7, 4, 'yes'}
Dictionary	No	No**	{ } or dict()	{'Jan': 75, 'Jul': 89}

25. Python Functions Theory:-

- Functions are way to divide your code.
- Improves code readability and reusability.
- Must be start with def keyword.
- Two types of functions

- Required arguments
- Default arguments

→ Ex:

```
def add_two_numbers(a, b):
    return a+b
```

26. Python Function Program:-

→ Open index.py

```
def add_two_numbers(a, b):
    return a+b
```

```
total = add_two_numbers(10, 10)
Print(total)
```

→ Output — python index.py

20

```
def add_two_numbers(a, b=10):
    return a+b
```

```
total = add_two_numbers(10, 20)
Print(total)
```

→ Output — python index.py

30

```
def add_two_numbers(a, b=10):
    return a+b
```

```
total = add_two_numbers(10)
Print(total)
```

→ Output → python index.py

20

→ def add_two_numbers (str):

return str

total = add_two_numbers (str="Tamil Tutorial")

Print (total)

→ Output - python index.py

Tamil Tutorial
:- manipal noitnaw noitpy .de

→ def add_two_numbers (str):

return str

total = add_two_numbers ("Tamil Tutorial")

Print (total)

→ Output → python index.py

Tamil Tutorial

→ def add_two_numbers (a,b):

Print (a+b)

add_two_numbers (10,10)

→ Output → python index.py

20

→ def add_two_numbers (a,b):

return a+b

total = add_two_numbers (10,10)

Print (total)

Print (type (total))

→ Output → python index.py

20
<class 'int'>

→ def add - two - numbers (a, b):

return a+b

total = add - two - numbers ("10", "10")

Print (total)

Print (type (total))

→ Output → python index.py

<class 'str'>

27. Python Exception Theory :-

→ Exceptions are the errors which we can handle

→ All the exception classes are must need to derive from Exception

→ Ex: Divide By Zero Exception
Stack Overflow Exception
and more.

→ How to handle Exception

→ Syntax :-

Try:

.....

Except To Error:

.....

→ We have finally block

→ We have else block

→ Syntax :-

Try:

Except:

Else:

28. Python Exception Program:-

-> Open index.py

-> Print (1/0)

-> Output - python index.py

Traceback (most recent call last):

File "index.py", line 2, in <module>

: print (1/0)

ZeroDivisionError: division by zero

-> try:

Print (1/0)

except ZeroDivisionError as ex:

Print ("handled " + str(ex))

-> Output :- python index.py

handled division by zero

-> #Print (1/0)

try:

Print (1/0)

except ZeroDivisionError as ex:

Print ("handled " + str(ex))

except IOError as ex:

Print ("handled " + str(ex))

except ImportError as ex:

```
Print ("handled " + str(ex))
```

except Exception:

```
Print ()
```

→ # Print (1/0)

try:

```
# Print (1/0)
```

```
Print ("im from try")
```

except Zero Division Error as ex:

```
Print ("handled " + str(ex))
```

finally:

```
Print ("Im from finally")
```

→ Output → python index.py

Im from try

Im from finally

→ # Print (1/0)

try:

```
# print (1/0)
```

```
Print ("im from try")
```

finally:

```
Print ("Im from finally")
```

→ Output → python index.py

Im from try

Im from finally

→ # print (1/0)

try:

```
Print (1/0)
```

```
# print ("im from try")
```

~~finally: ("I~~

finally:

Print ("Im from finally")

→ Output → python index.py

Im from finally

Traceback (most recent call last):

File "index.py", line 5, in <module>

Print (1/0)

ZeroDivisionError: division by Zero

→ try:

Print (1/0)

Print ("im from try")

except ZeroDivisionError as ex:

Print ("Handled" + str(ex))

else:

Print ("I'm from else")

finally:

Print ("Im from finally")

→ Output → python index.py

Im from try

I'm from else

Im from finally

→ try:

Print (1/0)

Print ("im from try")

```

except ZeroDivisionError as ex:
    print("Handled" + str(ex))
else:
    print("I'm from else")
finally:
    print("I'm from finally")

```

→ Output → python index.py

Handled division by zero

I'm from finally

29. Python Custom Exceptions:-

```

→ Open index.py
→ Class Application Custom Exception (Exception):
    pass

Class ValidationException (Application Custom Exception):
    pass

try:
    if (0 < 10):
        raise ValidationException
except ValidationException as ex:
    print("exception raise")

```

→ Output → python index.py

Exception raise

30. Python Import Tutorial:-

→ Open index.py


```
→ class Mathematics:  
    def add_two_numbers(self, a, b) → int:  
        return a+b
```

```
→ Open index.py  
import from index import Mathematics
```

```
a = Mathematics()
```

```
Print(a.add_two_numbers(10, 10))
```

```
→ Output - python index2.py python .py  
20
```

31. Python Class Basics Tutorial :-

→ classes are nothing but the template

→ Objects are the containers

→ Object has the class members and member functions.

→ Syntax :

```
Class : ClassName:
```

```
    Pass
```

→ object creation

```
obj = ClassName():
```

32. Python Static Method Tutorial :-

→ Open index.py

```
→ class StaticClassDemo
```

```
    def __init__(self):
```

```
        pass
```

@ Static method

```
def testMethod(self):  
    print("from method")
```

33. What is Static Class Demo?

a. testMethod()

→ Output — python index.py
from method

→ class StaticClassDemo(object):
 def __init__(self):

pass

@ Static method

```
def testMethod():  
    print("from method")
```

a = StaticClassDemo()

a.testMethod()

→ Output → python index.py
from method

→ class StaticClassDemo(object):

```
def __init__(self):  
    pass
```

@ Static method

```
def testMethod():  
    print("from method")
```

@ classmethod

```
def testMethod(cls):  
    print("from class method")
```

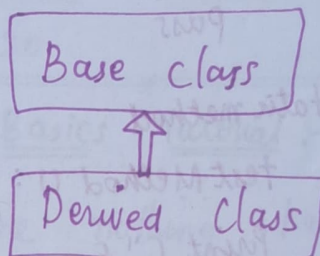
Static Class Demo. test method 1(c)

→ Output - python index.py

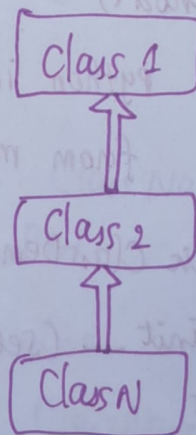
from class method

33. What is inheritance in python?

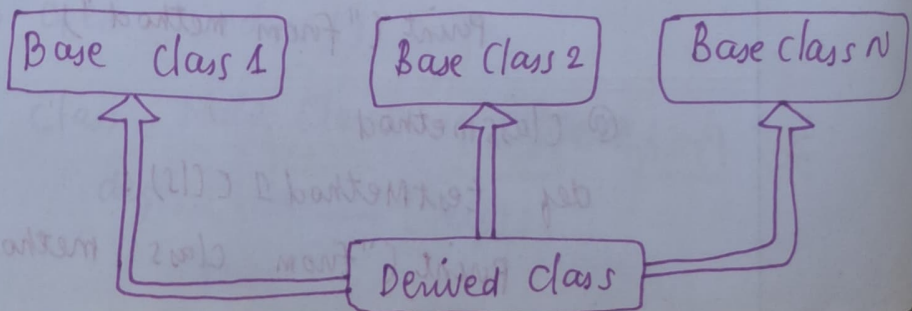
- Inheritance is an important aspects
- It provides code reusability.
- Concepts are Base class and derived class.
- Derived class acquires properties from the base class.



→ Multi level inheritance



→ Multi level inheritance



34. Python Inheritance Program Part -1 :-

→ Open index.py

→ class vehicle :

```
def engine(self):  
    print('From base class Engine')
```

```
class Car(vehicle):
```

```
    def brake-system(self):
```

```
        print('From the derived class')
```

```
v = vehicle()
```

```
v.engine()
```

```
c = Car()
```

```
c.brake-system()
```

→ Output : — python index.py

From base class Engine
from the derived class

→ class vehicle :

```
    def engine(self):
```

```
        print('From base class Engine')
```

```
class Car(vehicle):
```

```
    def brake-system(self):
```

```
        print('from the derived class')
```

```
c = Car()
```

```
c.brake-system()
```

```
c.engine()
```

∴ D-→ Output → python index.py part 9 .48

from the derived class
From base class Engine

35. Python Inheritance Program part -2 :-

→ Open index.py

→ class vehicle:

def engine(self):

Print('From base class Engine')

class Car(vehicle):

def brake-system(self):

Print('from the derived class')

class mini-Car(Car):

def mini-brake-system(self):

Print('mini Car')

c = mini-Car()

c.brake-system()

c.engine()

c.mini-brake-system()

→ Output → python index.py

from the derived class

From base class Engine

mini Car

→ class vehicle:

def engine(self):

```
print('From base class Engine')
```

```
class Car:
```

```
def brake-system(self):
```

```
    print('from the derived class')
```

```
class mini-car(Vehicle, Car):
```

```
def mini-brake-system(self):
```

```
    print('mini car')
```

```
c = mini-car()
```

```
c.brake-system()
```

```
c.engine()
```

```
c.mini-brake-system()
```

→ Output — python index.py

from the derived class

from base class Engine

mini car

36. How to access Private members in Python?

→ Open index.py

```
→ class Vehicle:
```

```
def engine(self):
```

```
    print('From base class Engine')
```

```
def _engines(self):
```

```
    print('from private method')
```

```
class Car(Vehicle):
```

```
def brake-system(self):
```

```
    print('from the derived class')
```


C = Car()

C.brake_system()

C.engine()

C.__engines()

→ Output — python index.py

Traceback (most recent call last):

File "index.py", line 15, in <module>

C.__engines()

AttributeError: 'Car' object has no attribute '__engines'

→ Class Vehicle:

def engine(self):

Print('From base class Engine')

def __engines(self):

Print('from private method')

class Car(Vehicle):

def brake_system(self):

Print('from the derived class')

Super().engine()

C = Car

C.brake_system()

→ Output → `class Car (Vehicle)` ^{once confirm this output}

File "index.py", line 12, in Car

`super (). engine ()`

Runtime Error : `super ()` : no arguments

→ `class Vehicle :`

`def __init__ (self):`

`print ("from base class constructor")`

`def engine (self):`

`print ('From base class engine')`

`def __engine (self):`

`print ('from Private method')`

`class Car (Vehicle):`

`def __init__ (self):`

`super (). __init__ ()`

`def brake_system (self):`

`print ('from the derived class')`

`c = Car ()`

`c. brake_system ()`

→ Output → `python index.py`

from base class constructor

from the derived class

→ Class Vehicle:

```
def __init__(self):
```

```
    print("from base class constructor")
```

```
def engine(self):
```

```
    print('From base class Engine')
```

```
def __engine(self):
```

```
    print('from private method')
```

```
class Car(Vehicle):
```

```
    def __init__(self):
```

```
        print('derived constructor')
```

```
    def brake_system(self):
```

```
        print('from the derived class')
```

```
c = Car()
```

```
c.brake_system()
```

→ Output → python index.py

derived constructor

from the derived class

→ class vehicle :

```
def __init__(self):
```

```
    print("from base class constructor")
```

```
def engine(self):
```

```
    print('From base class Engine')
```



```
def __engine__(self):
```

```
    print('from private method')
```

```
class Car(Vehicle):
```

```
    def brake_system(self):
```

```
        print('from the derived class')
```

```
c = Car()
```

```
c.brake_system()
```

→ Output → python index.py

from base class constructor
from the derived class

→ class Vehicle:

```
    def engine(self):
```

```
        print('From base class Engine')
```

```
    def __engine__(self):
```

```
        print('from private method')
```

```
class Car(Vehicle):
```

```
    def brake_system(self):
```

```
        print('from the derived class')
```

```
c = Car()
```

```
c.brake_system()
```

→ Output — python index.py

from the derived class

→ class Vehicle:

```
    def engine(self):
```

```
Print ('From base class Engine')
```

```
def _engines (self):
```

```
Print ('from private method')
```

```
class Car (Vehicle):
```

```
def brake_system (self):
```

```
Print ('from the derived class')
```

```
C = Car()
```

```
C.brake_system()
```

```
C.Engine()
```

→ Output: — python index.py

from the derived class
From base class Engine

→ class Vehicle :

```
def engine (self):
```

```
Print ('From base class Engine')
```

```
def _engines (self):
```

```
Printf ('from private method')
```

```
class Car (Vehicle):
```

```
def brake_system (self):
```

```
Print ('from the derived class')
```

```
v = Vehicle()
```

V. — engines()

C = car()

C.bruke_system()

C. — engines()

→ Output — python index.py

Traceback (most recent call last):

File "index.py", line 13, in <module>

V. — engines()

AttributeError: 'vehicle' object has no attribute '_engines'.

→ Class Vehicle:

def engine(self):

Print('From base class Engine')

Self. — engines()

def — engines(self):

Print('from private method')

class car(Vehicle):

def bruke_system(self):

Print('from the derived class')

C = car()

C.bruke_system()

C. engine()

- Output — python index.PY
- from the derived class
 - from base class Engine
 - from private method

37. Python File Operations:-

- python also supports file operations as like other programming languages.
- No need to import any third party library to do so.
- Python provides unbuild function itself
- File Creation, open and read all the operations are based on the file mode
- Available file modes:-
 - W - writing
 - R - Reading
 - A - Appending
 - R+ - Both read and write
- Also can use With for file operation.
- Need to memorize only few methods alone
 - open()
 - close()
 - Write()
 - Read()
- File operation are slow always. Keep in mind.

38. Python File Operation Program Part-1 :-

→ Open index.py

```
f = open('tutorial.txt', 'w')  
f.close()
```

→ Output — python index.py

Can create tutorial.txt from
index.py

→ try:

```
f = open('tutorial.txt', 'w')
```

```
f.write('this is from python tamil tutorial')
```

finally:

```
f.close()
```

→ Output → python index.py

Can ~~open~~ ^{open} tutorial.txt can
view on this is from python
tamil tutorial.

→ try:

```
f = open('tutorial.txt', 'w')
```

```
f.write('this is the new content to  
the file')
```

finally:

```
f.close()
```

→ Output → python index.py

Can open tutorial.txt can
view on this is from python
tamil tutorial.

```
f = open('tutorial.txt', 'a')
```

```
f.write('this is the new content to  
the file')
```

finally:

```
f.close()
```

→ Output — python index.py

Can open tutorial.txt and
see on this is the new
content to the file this is the
new content to the file.

→ try:

```
f = open('tutorial.txt', 'r')
```

```
f.write('this is the new content to  
the file')
```

finally:

```
f.close()
```

→ Output — python index.py

Traceback (most recent call last):

File "index.py", line 3, in <module>

```
f.write('this is the new  
content to the file')
```

io.UnsupportedOperation: not writable

39. Python File Operation Program Part - 2 :-

→ Open index.py

try:

```
f = open('tutorial.txt', 'r')
```

```
Print (f.read())
```

finally:

```
f.close()
```

→ Output — python index.py

this is the new content to the file
this is the new content to the file.

→ try:

```
f = open('tutorial.txt')
```

finally:

```
f.close()
```

→ Output - Python index.py

can empty

→ try:

```
f = open('tutorial.txt')
```

```
f.write('test write')
```

finally:

```
f.close()
```

→ Output - python index.py

Traceback (most recent call last):

File "index.py", line 3, in <module>

f.write('test write')

io.UnsupportedOperation: not writable

3. Python File Operation Part - 2

→ try:

with open('tutorial.txt') as f:

f.write('test write')

finally:

f.close()

→ Output — python index.py

Traceback (most recent call last):

File "index.py", line 3, in <module>

f.write('test write')

IOError: [Errno 13] Unsupported Operation: not writable

→ try:

with open('tutorial.txt', 'w') as f:

f.write('test write')

finally:

f.close()

→ Output → python index.py

Can open: tutorial.txt can
see test write.

Content of tutorial.txt
test write

→ try:

with open('tutorial.txt', 'r+') as f:

f.write('test write')

print(f.read())

finally:

f.close()

→ Output → pytho Index.py

test write on tutorial.txt

40. python Database Theory:-

→ python supports Database operations as like any other programming languages

→ python has inbuilt sqlite 3 DB within it.

→ For any other DB we need to install the package.

→ Databases are nothing but the collection of Tables.

→ Tables are the collections on Rows and Columns

→ DBMS VS RDBMS

→

	Id	Name	Salary	depart-ment	Position	hire Date
1	Filter	Filter	Filter	Filter	Filter	Filter
	1	John	700.0	HR	Manager	2017-01-04

→ DML Statements

→ DDL Statements

→ Transaction related Statements

→ Data types :

→ Varchar / text

→ Integer / number

41. python Database Practical Part - I :-

→ Open ~~index~~ index.py

→ import sqlite3

→ con = sqlite3.connect('student.db')

→ Output :- python index.py

* Student.db Create on index.py

→ import sqlite3

con = sqlite3.connect('student.db')

cursor_obj = con.cursor()

sql_query = "Create table Student (id integer
Primary Key, name text)"

cursor_obj.execute(sql_query)

~~cursor_obj.com~~

con.commit()

→ Output → python index.py

No data in display

(No + display any element)

42. Python SQLite Database Practical Part-2 :-

→ Open index.py

→ import sqlite3

con = sqlite3.connect('student.db')

cursor_obj = con.cursor()

sql_query = "Create table Student

(id integer Primary Key, name
text)"

```

# Cursor - obj. execute (Sql - query)
Sql - insert = "insert into student (id, name) value
                (?, ?)"
Cursor - obj. execute (Sql - insert, (2, 'Kumaran'))

Sql - select = "select * from student"
Cursor - obj. execute (Sql - select)
datas = cursor - obj. fetchall()
for data in datas:
    Print (data)
Con. commit

```

→ Output → python index.py

```

(1, 'Gopi')
(2, 'Kumaran')

```

43. Python SQLite Database Practical Part-3:-

```

→ Open index.py
→ import sqlite3

def get - connection ():
    return sqlite3.connect ('students.db')

def create - table (con):
    cur - obj = con.cursor()
    Sql - query = "create table student (id integer
                  Primary Key, name text)"
    cur - obj. execute (Sql - query)
    con. commit ()

def insert - record (con, values):
    cur - obj = con.cursor()

```



```
sql_insert = "insert into student (id, name)  
Values (?, ?)"
```

```
ins - cur - obj.execute(sql_insert, values)  
con.commit()
```

```
def fetch_record(con):
```

```
    cursor_obj = con.cursor()
```

```
    sql_select = "Select * from student"
```

```
    cursor_obj.execute(sql_select)
```

```
    datas = cursor_obj.fetchall()
```

```
    for data in datas:
```

```
        print(data)
```

```
    con.commit()
```

```
conn = get_connection()
```

```
# create_table(conn)
```

```
insert_record(conn, (2, 'Kumaran'))
```

```
fetch_record(conn)
```

→ Output → python inden.py

(1, 'Gopi')

(2, 'Kumaran')

(3, 'Anun')

44. Python Abstract Class :-

→ It's always a base class

→ It's uncomplete. So, cant create object of it.

- Python won't support it in straight forward.
- We need to use module called ABC
- Derived class must provide definition for the abstract method.
- All the abstract class must inherit from the ABC class.
- All the methods must be declared with the attribute abstract method

@ abstractmethod

def method(self):

pass

45. Python Abstract Class Program :-

- Open: ABC.py → file.py
- from abc import ABC, abstractmethod

class abc-class(ABC):

@ abstractmethod

def method(self):

print('from ABC class')

class and-class:

def method1(self):

print('from method 1')

- Output : python ABC.py

- from abc import ABC, abstractmethod
- class abc-class(ABC):
- @ abstractmethod
- def method(self):
- pass

```
class ord-class (abc-class):
```

```
def method (self):
```

```
print ('from method 1')
```

→ Output : python ABC.py

→ from abc import ABC, abstractmethod

```
class abc-class (ABC):
```

```
@ abstractmethod
```

```
def method (self):
```

```
pass
```

```
class ord-class (abc-class):
```

```
def method 1 (self):
```

```
print ('from method 1')
```

```
def method (self):
```

```
print ('base method')
```

```
a = ord-class()
```

```
a.method 1()
```

→ Output → python ABC.py

from method 1

→ from abc import ABC, abstractmethod

```
class abc-class (ABC):
```

```
@ abstractmethod
```

```
def method (self):
```

```
print ('from ABC class')
```

```
class ord-class (abc-class):
```

```
def method 1 (self):
```

```
print ('from method 1')
```



```
def method (self):
```

```
    print ('base method')
```

```
a = ord - class()
```

```
a . method 1()
```

→ Output → python ABC . py
from method 1

→ from abc import ABC, abstractmethod

```
class abc - class (ABC):
```

```
    @ abstract method
```

```
    def method (self):
```

```
        print ('from ABC class')
```

```
class ord - class (abc - class):
```

```
    def method 1 (self):
```

```
        print ('from method 1')
```

```
    def method (self):
```

```
        print ('base method')
```

```
a = ord - class()
```

```
a . method 1()
```

```
a . method ()
```

→ Output → python ABC . py

from method 1

base method

→ from abc import ABC, abstractmethod

```
class abc - class (ABC):
```

```
    @ abstract method
```

```
    def method (self):
```



```

Pass
class ord-class(abc-class):
    def method 1(self):
        print('from method 1')
    def method (self):
        print('base method')

```

a = ord-class()

a.method 1()

a.method()

→ Output → Python ABC.Py
from method 1
base method.

46. Python : MySQL Database Practical :-

- Download → ~~workbench~~ [MySQL workbench]
- Install XAMPP
- Open MySQL connections
- then click ⊕ symbol.
- Type name
- Then click rectangle box show bottom of the MySQL connections.
- Then double click open MySQL workbench
- File → Query 1 → Can create the file.
- Open Query 1
- Create table Student Cid int primary key auto-increment, name varchar

```

insert into student (name) values ('Gopi');
insert into student (name) values ('programming');
select * from student

```

→ Output →

id	name
1	Gopi
2	programming
Null	NULL

- Open Visual Studio Code
- And open mysql.py file (new)
- Then type -
- Then create file db-sql.py.
- Then Run db-sql.py
- import mysql.connector

```

con = mysql.connector.connect (host='local
                                host', user="root", pass
                                word="", database="test")

```

```

cur = con.cursor()

```

```

cur.execute ("select * from student")

```

```

result = cur.fetchall()

```

```

for i in result:

```

```

    print(i)

```

→ Output — python db-sql.py

```

(1, 'Gopi')

```

```

(2, 'programming')

```

47. python Date & Time :-

→ Open index.py

→ from datetime import datetime
from datetime import date

Print (datetime.now())

→ Output — python index.py

2020-05-10 17:06:07.693705

→ from datetime ~~import~~ datetime
from datetime import date

Print (datetime.now())

Print (date.today())

→ Output — python index.py

2020-05-10 17:08:50.884039

2020-05-10

→ from datetime import datetime
from datetime import date

Print (datetime.now())

Print (date.today())

Print (date.today().strftime('%d/%m/%y'))

→ Output — python index.py

2020-05-10 17:10:17.547996

2020-05-10

10/05/20

→ from ~~datetime~~ date import ~~datetime~~ date
from datetime import date

Print (datetime.now())

Print (date.today())

Print (date.today().strftime('%B-%m-%Y'))

→ Output — python index.py

2020-05-10 17:11:23.716781

2020-05-10

May -05-20

→ from datetime import datetime

from datetime import date

Print (datetime.now())

Print (date.today())

Print (date.today().strftime('%B-%m-%Y'))

a = datetime.strptime('2020-05-10 17:11:23.

716781', '%B-%m-%Y')

Print(a)

→ Output → python index.py

(data-string, format))

ValueError: time data '2020-05-10

17:11:23.716781' does not

match format '%B-%m-%Y'

Can use a = datetime.strptime('20-05-10', '%g-%m-%d')

Print(a)

→ Output — python index.py

2020-05-10 00:00:00

→ from datetime import datetime
from datetime import date

print (datetime.now())

print (date.today())

print (date.today().strftime('%d-%m-%y'))

a = datetime.strptime('20-05-10', '%d-%m-%y')

print(a)

print(type(a))

→ Output → python index.py

2020-05-10 00:00:00

<class 'datetime.datetime'>

48. Python Regex Theory:-

→ Regular expression is for validation

→ Python has re module

→ With match method we can compare the data with pattern.

→ Ex pattern: [0-9], [a-d], [^0-9]

→ Code Sample:

```
import re
```

```
pattern = '^a...a$'
```

re.match(pattern, data) → return boolean

49. Python Regex Practical Program:-

→ Open index.py


```

→ import re
data = "python tutorial"
Pattern = '^a....a$'
isValid = re.match(Pattern, data)
if (isValid):
    Print('success')
else:
    Print('failure')

```

→ Output — python index.py
failure

Can use data = "python"
Pattern = '^p....n\$'
Output → success

→ Open website Regular Expression. info can visit and read some.

50. python Iterator:-

→ Iterators are object that iterate upon

→ Base is — iter — and — next —

→ Iterators are everywhere in python

→ It's base for for loop.

→ list, tuples strings are iterable means, can use in loop

→ If you want to create your own iterators you can do so.

→ Ex: for loop iterator

→ Open notepad and then type for (just learning) list a = [1, 2, 3]

for i in a:

Print i

Obj = iter(a)

while true:

a = next(obj)

yield a

→ list a = [1, 2, 3]

for i in a:

print i

Obj = iter(a)

try:

while true:

a = next(obj)

yield a

except:

break

57. Python Decorator :-

→ Design ~~decorator~~ pattern

→ Adding additional functionality to an Object.

→ Reduce Code duplicate

→ python provide easy syntax for decorator

-s foo

→ Decorators are like a extension methods

→ which can accepts arguments too

→ Decorators executes first

→ Ex:

```
def to_uppercase (function):
```

```
def wrapper():
```

```
    return function().upper()
```

```
    return wrapper
```

52. Python Decorator Program:-

→ Open index.py

→ def upper_decorator (function):

```
    def wrapper (function):
```

```
        return function().upper()
```

```
    return wrapper
```

@upper_decorator

```
def getname():
```

```
    return "python tutorial"
```

```
Print (getname())
```

→ Output — python index.py

PYTHON TUTORIAL

→ def upper_decorator (function):

```
    def wrapper (a)
```

```
        Print ('from decorator', str(a))
```

```
    return function (a).upper()
```

```
    return wrapper
```

@upper_decorator

```
def getname (name):
```

```
    return "python tutorial"
```

```
Print (getname ('gopi'))
```


→ Output — Python_index.py

from decorator gopi

PYTHON TUTORIAL

53. Python Map & Zip Theory:-

→ Zip function take iterables

→ Returns iterator of Tuples

→ Zip accepts parameters of iterable

→ if empty argument, returns empty iterator

→ If passes different count arguments then it stops at the minimum value.

→ Map function accepts a function as first argument.

→ Second argument would be a set of values.

→ Map also supports lambda expression

→ Second parameter must be a iterable

54. Python Map & Zip Program:-

→ Open index.py

→ # Zip

a = Zip ([1,2,4,6], [2,3,4,6])

Print (a)

→ Output — Python_index.py

<Zip Object at 0x004DAB20>


```
-> # Zip
a = Zip ([1, 2, 4, 4], [2, 3, 4, 6])
result = Set (a)
Print (result)
```

```
-> Output — python index.py
{(1, 2), (4, 4), (2, 3), (4, 6)}
```

```
-> # Zip
a = Zip ([1, 2, 4, 4], [2, 3, 4, 6])
result = tuple (a)
Print (result)
```

```
-> Output — python index.py
[(1, 2), (2, 3), (4, 4), (4, 6)]
```

```
-> # map
def add-numbers (a, b):
    return a+b
a = map (add-numbers, [1, 2, 3, 4], [4, 5, 6, 3])
Print (a)
```

```
-> Output — python index.py
<map object at 0x00AED08D05>
```

```
-> # map
def add-numbers(a, b):
    return a+b
a = map (add-numbers, [1, 2, 3, 4], [4, 5, 6, 3])
result = tuple (a)
print (result)
```

→ Output — python index.py

(5, 7, 9, 7)

→ # map

```
def add-numbers(a):
```

```
    return a*a
```

```
a = map (add numbers, [1, 2, 3, 4])
```

```
result = tuple (a)
```

```
Print (result)
```

→ Output — python index.py

(1, 4, 9, 16)

→ # map

```
def add-numbers(a):
```

```
    return a*a
```

```
b = map (lambda a: a*a, [1, 2, 3, 4])
```

```
result = tuple (b)
```

```
Print (result)
```

→ Output — python index.py

(1, 4, 9, 16)

→ # map

```
def add-numbers(a):
```

```
    return a*a
```

```
b = map (lambda a, c: a*c, [1, 2, 3, 4], [4, 5, 2, 1, 6])
```

```
result = tuple (b)
```

```
print (result)
```


→ Output → python index.py
(45, 4, 3, 24)

→ # map

```
def add-numbers(a):
```

```
    return a*a
```

```
b = map(lambda a,c: a*c, [1,2,3,4], [45,2])
```

```
result = tuple(b)
```

```
Print(result)
```

~~→ python index.py~~
~~(45, 4)~~ → Output

→ Output — python index.py
(45, 4)

55. Python Job Opportunities :-

→ More job opportunity

→ python increases productivity → due to less no. of lines

→ Easy to learn

→ Massive amount of packages/modules/library to support all kinds of development.

→ ML, AI, Data science and many automations are relies on python. Of course we have other language support as well.

→ Fresher feels easy to learn. Not only fresher as well.

→ Good to learn more programming language.

→ Attracts Employer, all project has python code now a days.

→ Good pay.